Custom Compartmental Models

NodeBasedModels.jl Vignette 05

Simon Frost

2026-03-29

Table of contents

Introduction

One of the central design principles of NodeBasedModels is the separation of disease structure from network structure. The `CompartmentalModel` type defines the epidemiological states and transitions — infection along edges, spontaneous recovery, waning immunity — while the `GraphNetwork` defines the contact pattern. This composable architecture means you can take any compartmental model and run it on any graph, using individual-based or pair-based approximations.

In this vignette we:

1. Review the built-in models (SIR, SIS, SEIR, SIRS)
2. Run the SIS model on a graph to see endemic equilibrium dynamics
3. Define a custom SIRV model with vaccination
4. Explore the SEIR model with an exposed class
5. Build a two-stage infectious period model for non-exponential duration

Setup

```
using NodeBasedModels
using Graphs
using Plots
```

```
Precompiling packages...
  22554.8 ms  ✓ NodeBasedModels
  1 dependency successfully precompiled in 56 seconds. 294 already precompiled.
```

Built-in models

NodeBasedModels provides four standard compartmental models. Each is a `CompartmentalModel` containing a list of `Compartments` (with an `infectious` flag) and `Transitions` (typed as `:infection` for transmission along edges or `:spontaneous` for individual-level events):

```
for (name, model) in [("SIR", sir_model()), ("SIS", sis_model()),
                      ("SEIR", seir_model()), ("SIRS", sirs_model())]
    println("── $name ──")
    println("  Compartments: ", [c.name for c in model.compartments])
    println("  Infectious:   ", model.infectious_compartments)
    println("  Transitions:")
    for t in model.transitions
        println("    $(t.from) → $(t.to)  rate=$(t.rate)  type=$(t.type)")
    end
    println()
end
```

```
── SIR ──
  Compartments: [:S, :I, :R]
  Infectious:   [:I]
  Transitions:
    S → I  rate=τ  type=infection
    I → R  rate=γ  type=spontaneous

── SIS ──
  Compartments: [:S, :I]
  Infectious:   [:I]
  Transitions:
    S → I  rate=τ  type=infection
```

```
    I → S  rate=γ  type=spontaneous

── SEIR ──
  Compartments: [:S, :E, :I, :R]
  Infectious:   [:I]
  Transitions:
    S → E  rate=τ  type=infection
    E → I  rate=σ  type=spontaneous
    I → R  rate=γ  type=spontaneous

── SIRS ──
  Compartments: [:S, :I, :R]
  Infectious:   [:I]
  Transitions:
    S → I  rate=τ  type=infection
    I → R  rate=γ  type=spontaneous
    R → S  rate=ε  type=spontaneous
```

The transitions encode the disease dynamics:

- SIR: $S \xrightarrow{\tau} I \xrightarrow{\gamma} R$ — infection is permanent
- SIS: $S \xrightarrow{\tau} I \xrightarrow{\gamma} S$ — recovered return to susceptible
- SEIR: $S \xrightarrow{\tau} E \xrightarrow{\sigma} I \xrightarrow{\gamma} R$ — latent period before infectiousness
- SIRS: $S \xrightarrow{\tau} I \xrightarrow{\gamma} R \xrightarrow{\varepsilon} S$ — temporary immunity

SIS on a graph

The SIS model is qualitatively different from SIR because recovered individuals return to the susceptible pool. Instead of the epidemic burning out, the system reaches an endemic equilibrium where infection persists indefinitely.

```
g = random_regular_graph(200, 6; seed=42)
net = GraphNetwork(g);
println("Graph: N=$(nv(g)), E=$(ne(g)), mean_degree=$(mean_degree(net))")
```

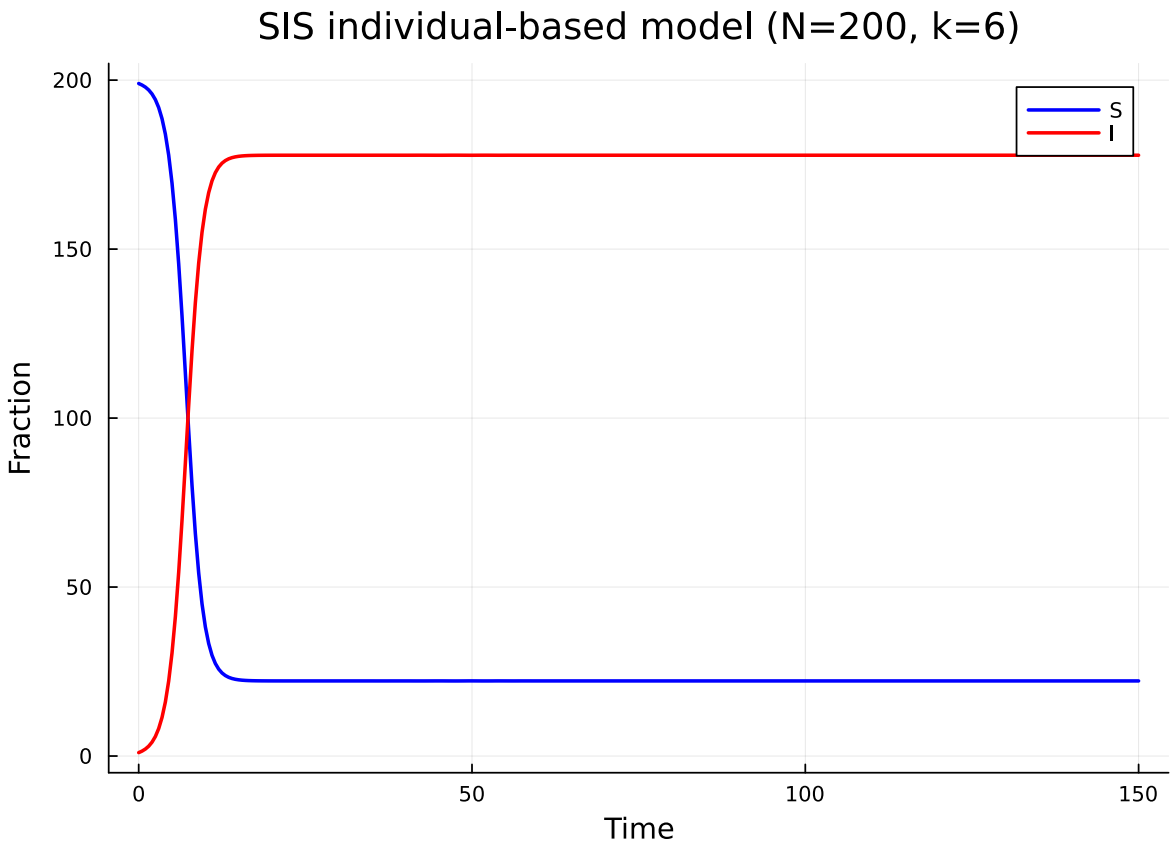
```
Graph: N=200, E=600, mean_degree=6.0
```

```
sis_result = generate_individual_based(sis_model(), net;
    infection_rate=0.15,
    recovery_rate=0.1,
    initial_infected=[1],
    tspan=(0.0, 150.0),
    saveat=0.5,
    ε=0.01
)
```

```
IndividualBasedResult(N=200, states=[:S], tspan=(0.0, 150.0))
```

For the SIS model, the individual-based approximation tracks $K = 1$ state (S only) per node, with I derived as $1 - S$:

```
S_sis = aggregate(sis_result, :S)
I_sis = aggregate(sis_result, :I)
t_sis = range(0.0, 150.0, length=length(S_sis))

plot(t_sis, S_sis, label="S", lw=2, color=:blue,
     xlabel="Time", ylabel="Fraction",
     title="SIS individual-based model (N=200, k=6)")
plot!(t_sis, I_sis, label="I", lw=2, color=:red)
```



Unlike SIR, the infected fraction does not decay to zero. Instead, $S(t)$ and $I(t)$ converge to an endemic equilibrium determined by the balance between infection and recovery on the network. The equilibrium prevalence depends on the ratio τ/γ relative to the epidemic threshold.

Custom SIR with vaccination

The `CompartmentalModel` constructor lets you define arbitrary compartmental structures. Here we build an SIRV model where susceptible individuals can be vaccinated at rate ν, moving to a protected class V:

$$S \xrightarrow{\tau} I, \quad I \xrightarrow{\gamma} R, \quad S \xrightarrow{\nu} V$$

```
sirv = CompartmentalModel(
    [Compartment(:S),
     Compartment(:I; infectious=true),
     Compartment(:R),
     Compartment(:V)],
```

```
    [Transition(:S, :I, :τ, :infection),
     Transition(:I, :R, :γ, :spontaneous),
     Transition(:S, :V, :ν, :spontaneous)];
    name = :SIRV
)
println("Compartments: ", [c.name for c in sirv.compartments])
println("Transitions:")
for t in sirv.transitions
    println("  $(t.from) → $(t.to)  rate=$(t.rate)  type=$(t.type)")
end
```

```
Compartments: [:S, :I, :R, :V]
Transitions:
  S → I  rate=τ  type=infection
  I → R  rate=γ  type=spontaneous
  S → V  rate=ν  type=spontaneous
```

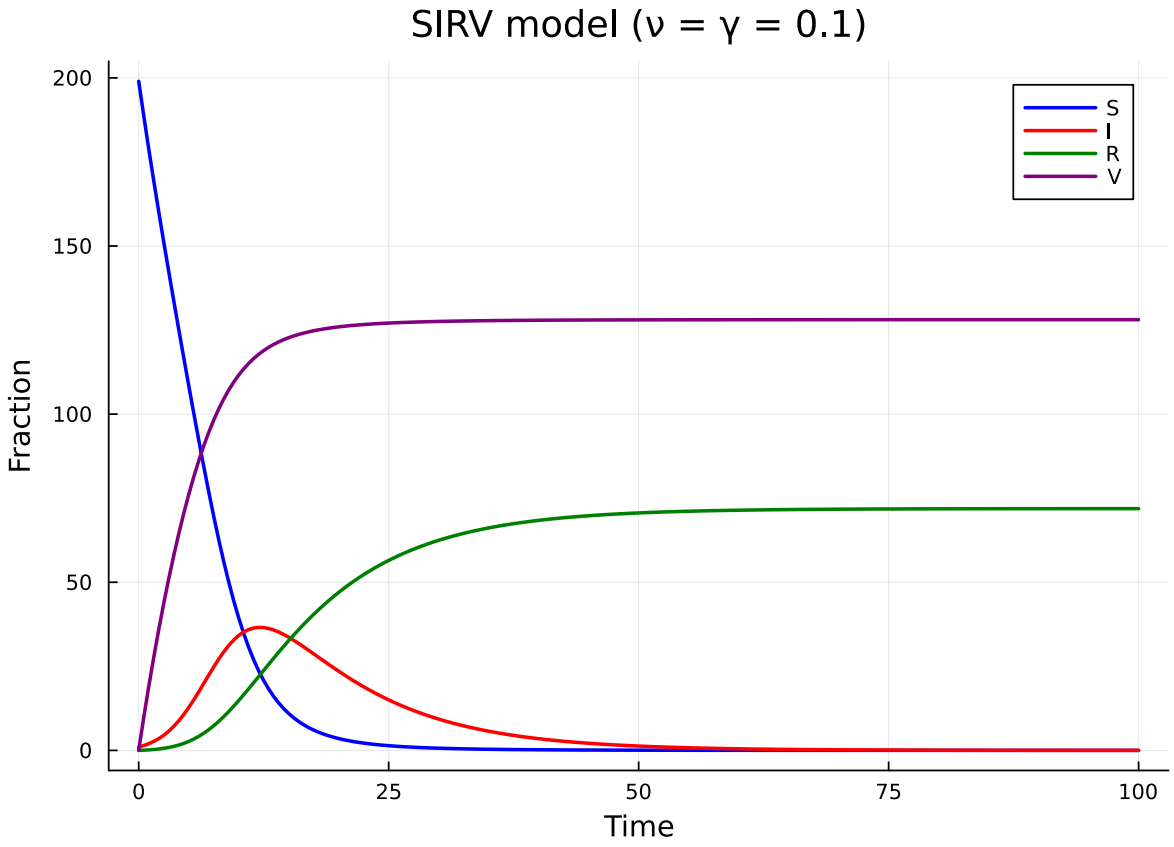
The model has two spontaneous transitions ($I \to R$ and $S \to V$). In the current `generate_individual_based` implementation, all `:spontaneous` transitions share the `recovery_rate` parameter. For a demonstration where the vaccination rate equals the recovery rate, we can run this directly:

```
sirv_result = generate_individual_based(sirv, net;
    infection_rate=0.15,
    recovery_rate=0.1,
    initial_infected=[1],
    tspan=(0.0, 100.0),
    saveat=0.5,
    ε=0.01
)
```

```
IndividualBasedResult(N=200, states=[:S, :I, :R], tspan=(0.0, 100.0))
```

```
S_v = aggregate(sirv_result, :S)
I_v = aggregate(sirv_result, :I)
R_v = aggregate(sirv_result, :R)
V_v = aggregate(sirv_result, :V)
t_v = range(0.0, 100.0, length=length(S_v))

plot(t_v, S_v, label="S", lw=2, color=:blue,
     xlabel="Time", ylabel="Fraction",
```

```
     title="SIRV model (ν = γ = 0.1)")
plot!(t_v, I_v, label="I", lw=2, color=:red)
plot!(t_v, R_v, label="R", lw=2, color=:green)
plot!(t_v, V_v, label="V", lw=2, color=:purple)
```



The vaccination pathway competes with infection: susceptibles are drained both by disease transmission and by vaccination. This reduces the peak prevalence and final size compared to standard SIR.

To model different vaccination and recovery rates, one would extend the rate mapping in `generate_individual_based` to accept a dictionary of rate symbols to values. This is a natural extension of the current framework.

SEIR model

The SEIR model introduces an exposed (latent) class E between susceptible and infectious:

$$S \xrightarrow{\tau} E \xrightarrow{\sigma} I \xrightarrow{\gamma} R$$

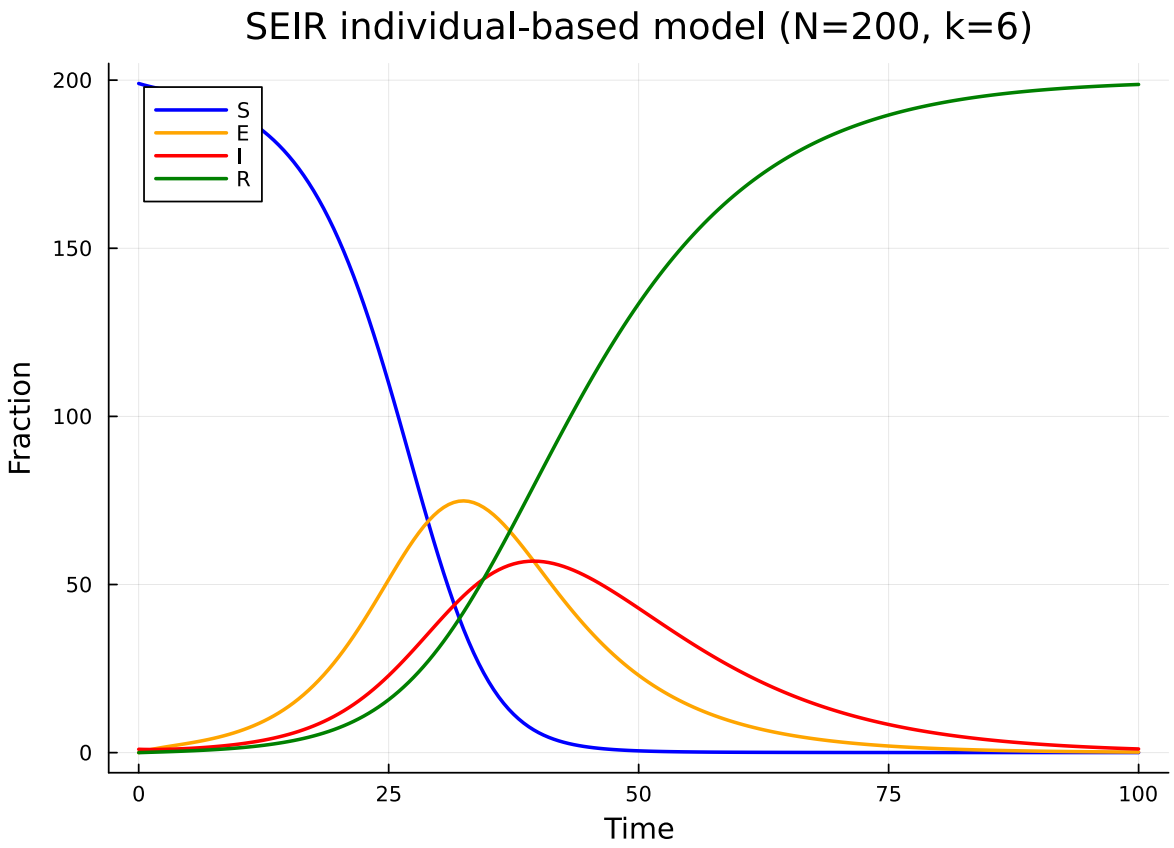
Individuals in E are infected but not yet infectious. This delays the epidemic peak and reduces its height compared to SIR.

```
seir_result = generate_individual_based(seir_model(), net;
    infection_rate=0.15,
    recovery_rate=0.1,
    initial_infected=[1],
    tspan=(0.0, 100.0),
    saveat=0.5,
    ε=0.01
)
```

```
IndividualBasedResult(N=200, states=[:S, :E, :I], tspan=(0.0, 100.0))
```

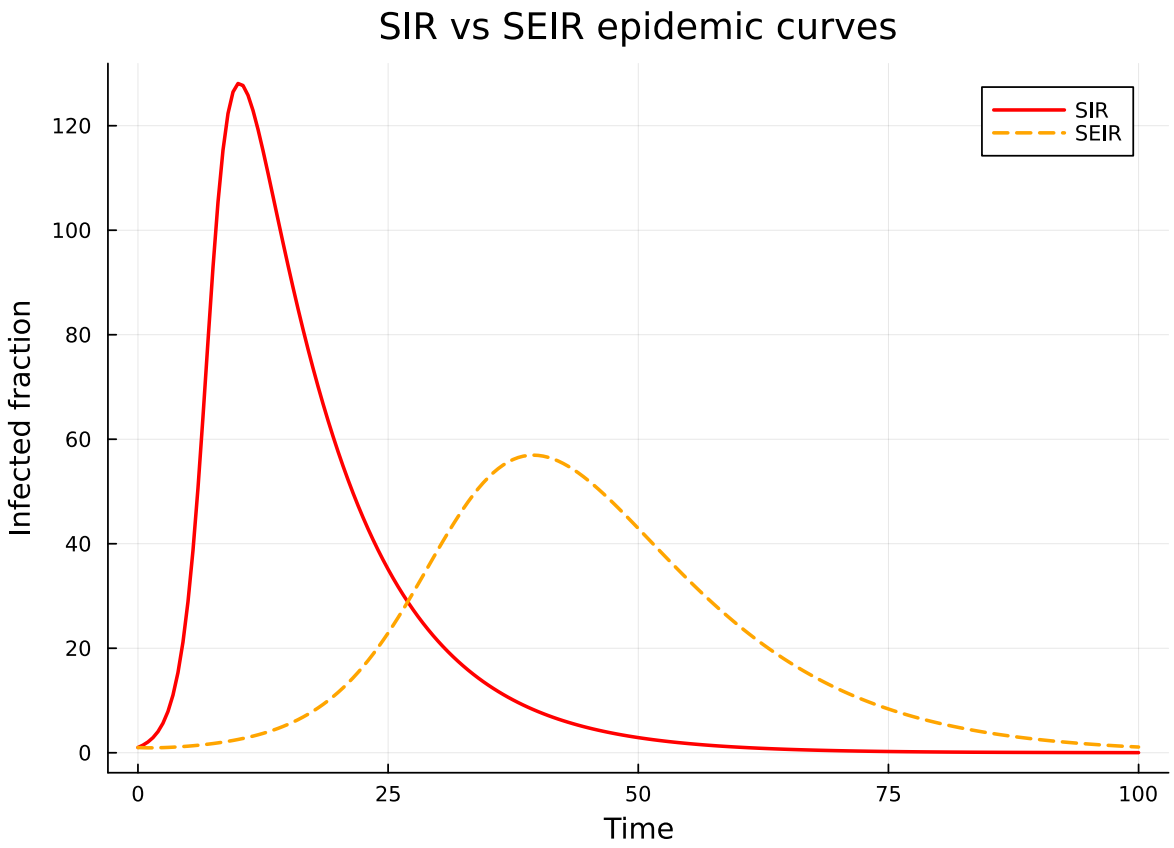
The SEIR model tracks $K = 3$ states (S, E, I) per node, with R derived as $1 - S - E - I$:

```
S_seir = aggregate(seir_result, :S)
E_seir = aggregate(seir_result, :E)
I_seir = aggregate(seir_result, :I)
R_seir = aggregate(seir_result, :R)
t_seir = range(0.0, 100.0, length=length(S_seir))

plot(t_seir, S_seir, label="S", lw=2, color=:blue,
     xlabel="Time", ylabel="Fraction",
     title="SEIR individual-based model (N=200, k=6)")
plot!(t_seir, E_seir, label="E", lw=2, color=:orange)
plot!(t_seir, I_seir, label="I", lw=2, color=:red)
plot!(t_seir, R_seir, label="R", lw=2, color=:green)
```



Compare the SIR and SEIR epidemic curves to see the delay and reduction caused by the latent period:

```
sir_result = generate_individual_based(sir_model(), net;
    infection_rate=0.15,
    recovery_rate=0.1,
    initial_infected=[1],
    tspan=(0.0, 100.0),
    saveat=0.5,
    ε=0.01
)
I_sir = aggregate(sir_result, :I)
t_sir = range(0.0, 100.0, length=length(I_sir))

plot(t_sir, I_sir, label="SIR", lw=2, color=:red,
     xlabel="Time", ylabel="Infected fraction",
     title="SIR vs SEIR epidemic curves")
plot!(t_seir, I_seir, label="SEIR", lw=2, ls=:dash, color=:orange)
```



The SEIR curve peaks later and lower because the exposed class acts as a buffer — newly infected individuals must pass through the latent period before contributing to transmission.

Two infectious stages

Real infectious periods are rarely exponentially distributed. A common technique to model Erlang-distributed durations is the method of stages: split the infectious compartment into n sequential stages, each with rate $n\gamma$, so the total sojourn time has an Erlang(n, γ) distribution.

Here we define an SIR model with two infectious stages:

$$S \xrightarrow{\tau} I_1 \xrightarrow{\sigma} I_2 \xrightarrow{\gamma} R$$

```
sir2 = CompartmentalModel(
    [Compartment(:S),
     Compartment(:I1; infectious=true),
     Compartment(:I2; infectious=true),
```

```
     Compartment(:R)],
    [Transition(:S, :I1, :τ, :infection),
     Transition(:I1, :I2, :σ, :spontaneous),
     Transition(:I2, :R, :γ, :spontaneous)];
    name = :SIR2
)
println("Model: ", sir2.name)
println("Infectious compartments: ", sir2.infectious_compartments)
```

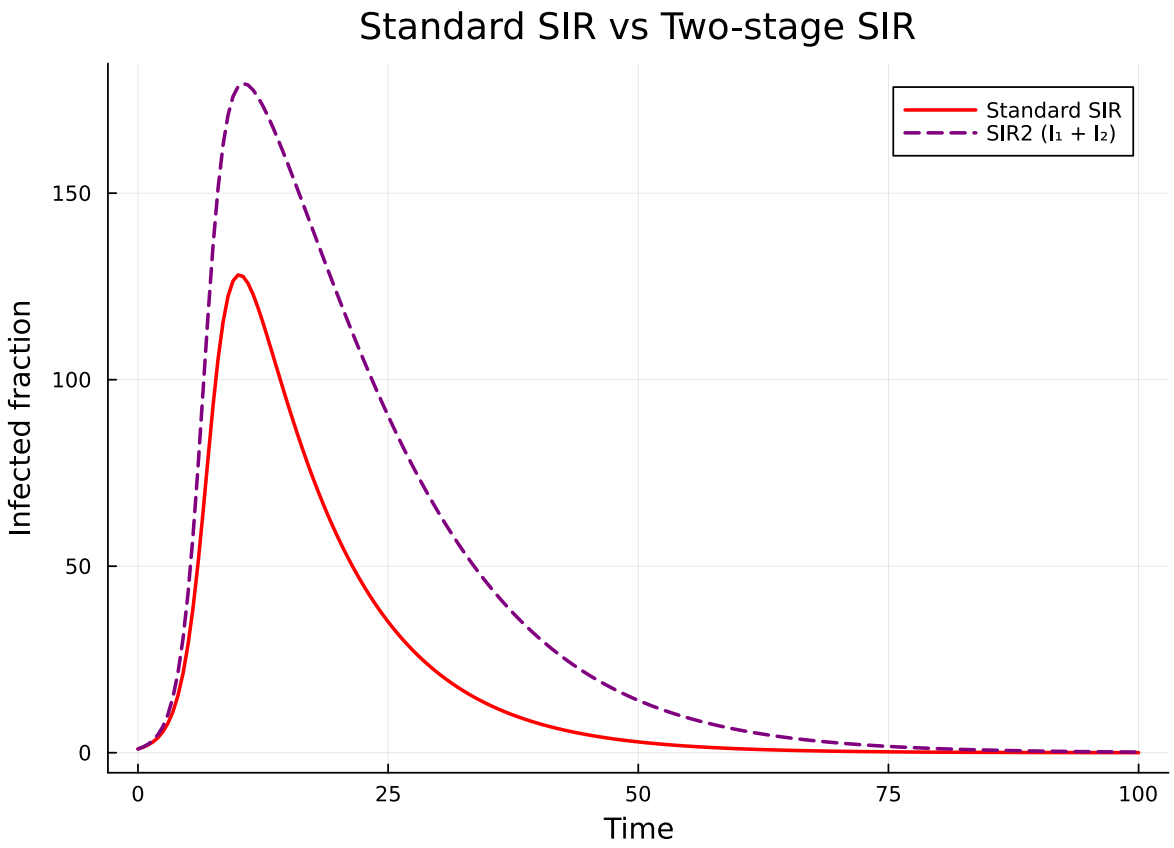
```
Model: SIR2
Infectious compartments: [:I1, :I2]
```

Both I_1 and I_2 are infectious (they can transmit), but only individuals in I_2 can recover. This creates a more peaked infectious period distribution.

```
sir2_result = generate_individual_based(sir2, net;
    infection_rate=0.15,
    recovery_rate=0.1,
    initial_infected=[1],
    tspan=(0.0, 100.0),
    saveat=0.5,
    ε=0.01
)
```

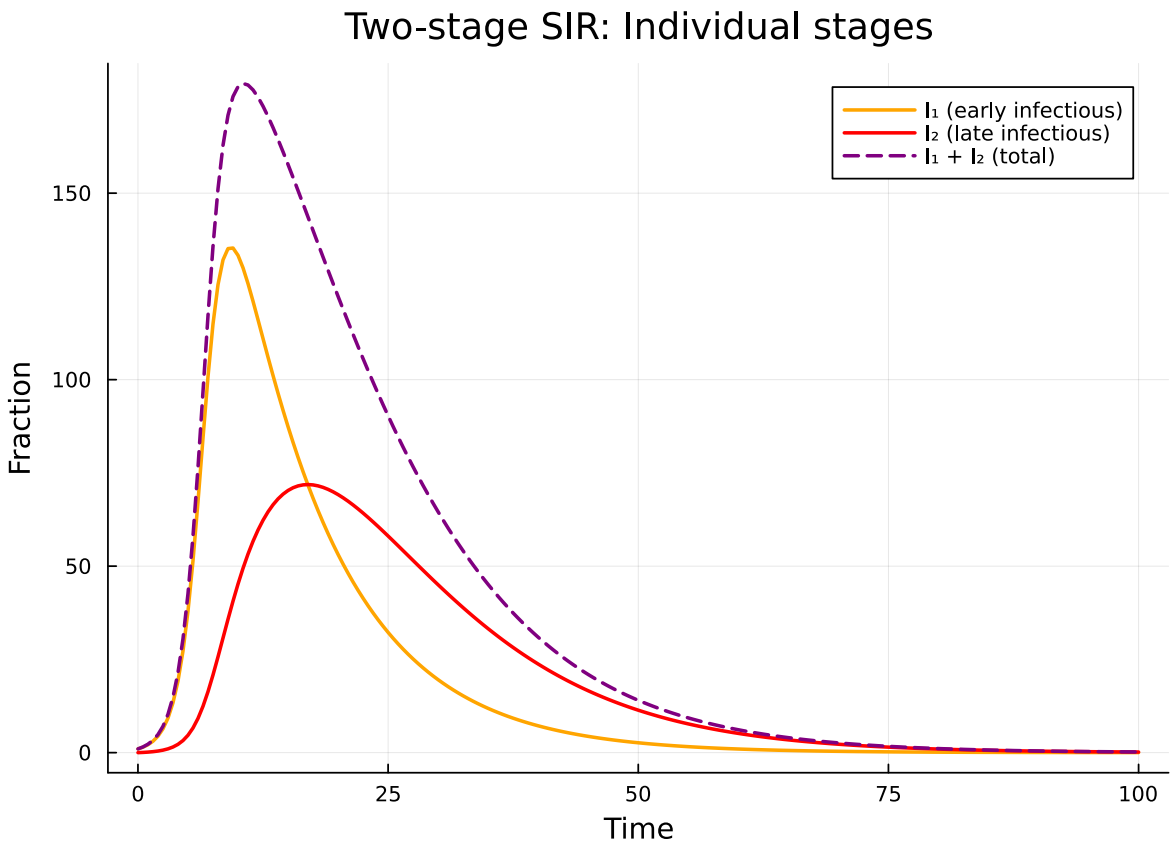
```
IndividualBasedResult(N=200, states=[:S, :I1, :I2], tspan=(0.0, 100.0))
```

```
I1_sir2 = aggregate(sir2_result, :I1)
I2_sir2 = aggregate(sir2_result, :I2)
t_sir2 = range(0.0, 100.0, length=length(I1_sir2))
I_total_sir2 = I1_sir2 .+ I2_sir2

plot(t_sir, I_sir, label="Standard SIR", lw=2, color=:red,
     xlabel="Time", ylabel="Infected fraction",
     title="Standard SIR vs Two-stage SIR")
plot!(t_sir2, I_total_sir2, label="SIR2 (I₁ + I₂)", lw=2, ls=:dash, color=:purple)
```



The two-stage model produces a sharper, more synchronised epidemic because the Erlang-distributed infectious period has lower variance than the exponential. Infected individuals progress through the stages at a more predictable pace, leading to a more concentrated wave of infection.

```
plot(t_sir2, I1_sir2, label="I₁ (early infectious)", lw=2, color=:orange,
     xlabel="Time", ylabel="Fraction",
     title="Two-stage SIR: Individual stages")
plot!(t_sir2, I2_sir2, label="I₂ (late infectious)", lw=2, color=:red)
plot!(t_sir2, I_total_sir2, label="I₁ + I₂ (total)", lw=2, ls=:dash, color=:purple)
```



Summary

The composable architecture of NodeBasedModels makes it straightforward to define and simulate custom compartmental models on arbitrary networks:

Model	Compartments	Transitions	Key behaviour
SIR	S, I, R	Infection, recovery	Epidemic burnout
SIS	S, I	Infection, recovery back to S	Endemic equilibrium
SEIR	S, E, I, R	Infection, progression, recovery	Delayed, reduced peak
SIRS	S, I, R	Infection, recovery, waning	Recurrent epidemics
SIRV	S, I, R, V	Infection, recovery, vaccination	Reduced final size

SIR2	S, I_1, I_2, R	Infection, staging, recovery	Sharper epidemic peak

The `CompartmentalModel` type requires only:

1. A list of `Compartments` with `infectious` flags
2. A list of `Transitions` typed as `:infection` (edge-based) or `:spontaneous` (node-based)

Any model defined this way can be passed to `generate_individual_based` or `generate_pair_based` and run on any `GraphNetwork`. This systematic separation of disease and network structure enables rapid exploration of how compartmental complexity interacts with contact heterogeneity.